

DS Lecture 2: Models of distributed systems

September 10th, 2025

Instructor: Elli Anastasiadi
ellia@cs.aau.dk

DEIS, Aalborg University, Denmark



AALBORG UNIVERSITY
DENMARK



About Elli:

Office: 1.2.57, Cassiopeia.

Feel free to send emails.

I love distributed systems

...



About Elli:

Office: 1.2.57, Cassiopeia.

Feel free to send emails.

I love distributed systems

...and math :)



About Elli:

Office: 1.2.57, Cassiopeia.

Feel free to send emails.

I love distributed systems

...and math :)

Our lectures:

1. Models of DS

3. Broadcast

5. Mutex

2. Time in DS

4. Consensus

6. Algorithms



About Elli:

Office: 1.2.57, Cassiopeia.

Feel free to send emails.

I love distributed systems

...and math :)

Our lectures:

1. Models of DS

3. Broadcast

5. Mutex

2. Time in DS

4. Consensus

6. Algorithms

- ▶ We will be using the white board some times.
- ▶ Exercises are meant to get you to think \ remember stuff.

All models are wrong, but some are useful

- *George Box*



AALBORG UNIVERSITY
DENMARK

Agenda



Aspects of models

General model for the course

Failures

Communication

Timing

Performance

What are the "important parts"?



What are the "important parts"?



Why make a system distributed?



What are the "important parts"?

Why make a system distributed?

- It is distributed by definition:



What are the "important parts"?

Why make a system distributed?

- **It is distributed by definition:** ...telecommunications...



What are the "important parts"?

Why make a system distributed?

- ▶ **It is distributed by definition:** ...telecommunications...
- ▶ **For reliability:** even if one node fails, the system keeps functioning.



What are the "important parts"?

Why make a system distributed?

- ▶ **It is distributed by definition:** ...telecommunications...
- ▶ **For reliability:** even if one node fails, the system keeps functioning.
- ▶ **Performance:** share load of computation, get data from nearby.



What are the "important parts"?

Why make a system distributed?

- ▶ **It is distributed by definition:** ...telecommunications...
- ▶ **For reliability:** even if one node fails, the system keeps functioning.
- ▶ **Performance:** share load of computation, get data from nearby.
- ▶ **To handle bigger problems:** Literally some problems have so much data they don't fit in one machine.

Not all distributed systems are the same



Not all distributed systems are the same



- How do they communicate?

Not all distributed systems are the same



- ▶ How do they communicate?
- ▶ How can they fail?

Not all distributed systems are the same



- ▶ How do they communicate?
- ▶ How can they fail?
- ▶ How do they compute?



Not all distributed systems are the same

- ▶ How do they communicate?
- ▶ How can they fail?
- ▶ How do they compute?
- ▶ What do they look like?

Modeling the processes - The API style



Idea: Each process/computer hosts a set of [modules](#) .



Modeling the processes - The API style

Idea: Each process/computer hosts a set of modules .
Each module provides an interface that accepts and produces events .



Modeling the processes - The API style

Idea: Each process/computer hosts a set of **modules** .

Each module provides an interface that accepts and produces **events** .

Different modules together, or one for every process should satisfy some **properties** .



Modeling the processes - The API style

Idea: Each process/computer hosts a set of modules .

Each module provides an interface that accepts and produces events .

Different modules together, or one for every process should satisfy some properties .

An event

$\{Event_type \mid Attributes, \dots\}$ or, $\{process_id \mid Event_type \mid Attributes, \dots\}$



Modeling the processes - The API style

Idea: Each process/computer hosts a set of modules .

Each module provides an interface that accepts and produces events .

Different modules together, or one for every process should satisfy some properties .

An event

$\{Event_type \mid Attributes, \dots\}$ or, $\{process_id \mid Event_type \mid Attributes, \dots\}$

A module

upon event $\{co \mid Event \mid att_1\}$ **such that** *condition* **do**
do something;

We want/will make:



We want/will make:



- ▶ APIs for modules and show that together they have some properties, and

We want/will make:



- ▶ APIs for modules and show that together they have some properties, and
- ▶ Implement the modules so that they satisfy some local properties.



We want/will make:

- ▶ APIs for modules and show that together they have some properties, and
- ▶ Implement the modules so that they satisfy some local properties.

In this part often: pseudo code + math to show that the above are not possible.

Types of Failure (abstraction)





Types of Failure (abstraction)

1. **crash-stop**: A process can crash, but all crashes can be reliably detected by everyone else. **These are the best** .
2. **crash-silent**: where process crashes can never be reliably detected,
3. **crash-noisy**: crashes can be detected some times (accuracy only eventually).
4. **crash-recovery**: where processes can crash and later recover and still participate (careful for **omissions**).
5. **crash-arbitrary**: where processes can deviate arbitrarily from the protocol specification and act in malicious, adversarial ways. Also called Byzantine failures. **These are the worst** .
6. (Extra, odd category) **randomized**. Where processes might make some choices. Correctness comes from probability theory (or not).

Q1: How many processes can be faulty before an algorithm fails?

Q2: Where does cryptography fit in all of this?

Links, networks, messages, and communication



Links, networks, messages, and communication



No internet

Try:

- Checking the network cables, modem, and router
- Reconnecting to Wi-Fi

`ERR_INTERNET_DISCONNECTED`

How do processes communicate?



Is communication actually needed?



How do processes communicate?

Is communication actually needed? If yes:

► **Message parsing:**

1. Types of links and their failure.
2. Topology (usually we assume fully connected).
3. Routing algorithms.
4. Broadcast.



How do processes communicate?

Is communication actually needed? If yes:

- ▶ **Message parsing:**

1. Types of links and their failure.
2. Topology (usually we assume fully connected).
3. Routing algorithms.
4. Broadcast.

- ▶ **Shared memory**

1. Who can view what location?
2. How do we guarantee that we are reading the "freshest" value?



On types of links

Links are modules that implement a send and a receive function (and satisfy some properties).

- ▶ TCP/IP: a pair of nodes can communicate, or not
- ▶ Ssh: guards corruption, interception, and other evil things.

On types of links

Links are modules that implement a send and a receive function (and satisfy some properties).

- ▶ TCP/IP: a pair of nodes can communicate, or not
- ▶ Ssh: guards corruption, interception, and other evil things.

Network failures - Types of links

1. Perfect: reliable delivery + no duplication + no creation.
2. Fair-loss: fair-loss + finite duplication + no creation.
3. Stubborn: stubborn delivery + no creation.
4. Logged-perfect: like perfect, but with logging.
5. Authenticated: reliable delivery + no duplication + authenticity.

Q3: Can you think of a behavior that is allowed by a **perfect link**, but might be problematic?

Do networks actually fail?



Thanks to TCP, and other lower-level protocols, we usually only deal with perfect links, and with fail-stop (i.e. detectable) crashes.



Do networks actually fail?

Thanks to TCP, and other lower-level protocols, we usually only deal with perfect links, and with fail-stop (i.e. detectable) crashes.

However:

Network Partitions



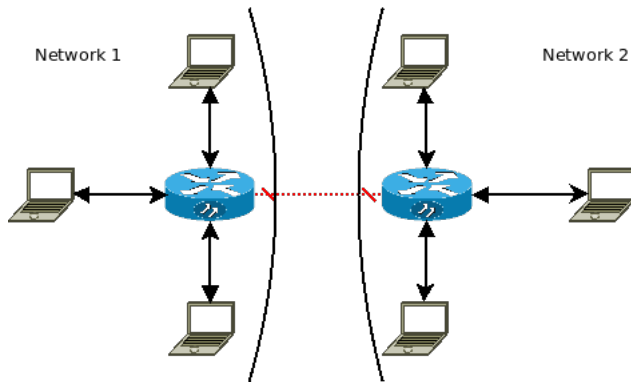
Do networks actually fail?

Thanks to TCP, and other lower-level protocols, we usually only deal with perfect links, and with fail-stop (i.e. detectable) crashes.

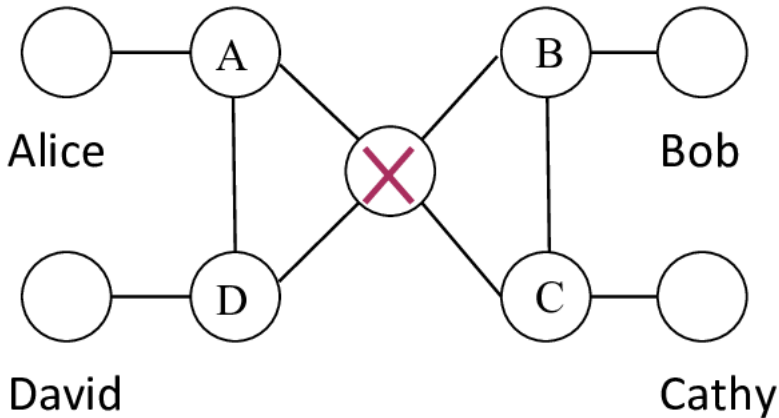
However:

Network Partitions Network partitions happen only (hopefully) when **many** links fail at the same time.

Examples on network partitions



Examples on network partitions



Crashes vs failures



Now that we are done with network and process failures.



Crashes vs failures

Now that we are done with network and process failures.
We say a **process** can [crash](#) . vs A **system** can [fail](#) .



Crashes vs failures

Now that we are done with network and process failures.

We say a **process** can [crash](#) . vs A **system** can [fail](#) .

For the whole system we assume (unless we say it) perfect links (thank you TCP/IP!).

Crashes vs failures

Now that we are done with network and process failures.

We say a **process** can [crash](#) . vs A **system** can [fail](#) .

For the whole system we assume (unless we say it) perfect links (thank you TCP/IP!).

For example: [fail-stop](#) =Processes can crash-stop + Perfect links.

Etc.

Timing assumptions



Synchronous vs. Asynchronous

Timing assumptions

Synchronous vs. Asynchronous

1. Asynchronous :

- ▶ No bound on any delays.
- ▶ No knowledge of different process speeds (relative).
- ▶ We use *logical clocks*.

2. Synchronous : will have:

- ▶ Timed failure detection.
- ▶ Measure of transit delays.
- ▶ Coordination based on time.
- ▶ Worst-case performance.
- ▶ Synchronized clocks (more on this later).

Timing assumptions

Synchronous vs. Asynchronous

1. Asynchronous :

- ▶ No bound on any delays.
- ▶ No knowledge of different process speeds (relative).
- ▶ We use *logical clocks*.

2. Synchronous : will have:

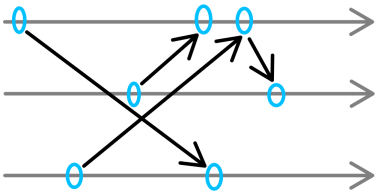
- ▶ Timed failure detection.
- ▶ Measure of transit delays.
- ▶ Coordination based on time.
- ▶ Worst-case performance.
- ▶ Synchronized clocks (more on this later).

Note: you might see these terms used differently. Keep an eye open.

Network Models

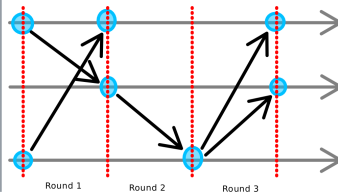
Asynchronous

- ▶ Arbitrary delays
- ▶ Unknown processing time



Synchronous

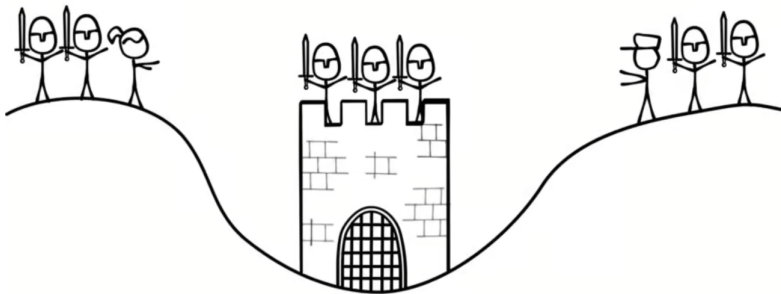
- ▶ Known delays
 - ▶ or hard limits of them
- ▶ Known drift



To be concrete - the two generals problem



To be concrete - the two generals problem



images from [linux blog.io](http://linux.blog.io)



Assume a fair loss link & an asynchronous system

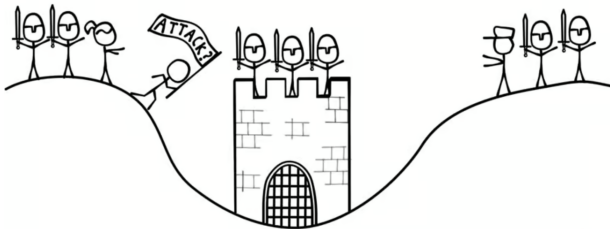


Assume a fair loss link & an asynchronous system

Can they agree on the time of attack?

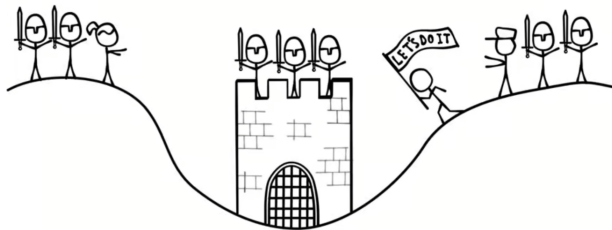
Assume a fair loss link & an asynchronous system

Can they agree on the time of attack?



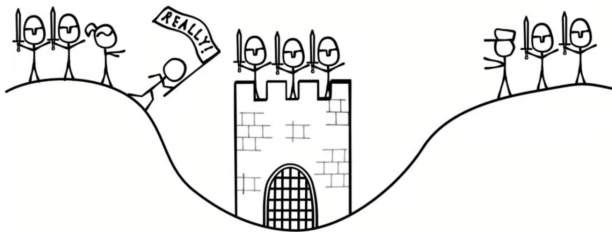
Assume a fair loss link & an asynchronous system

Can they agree on the time of attack?



Assume a fair loss link & an asynchronous system

Can they agree on the time of attack?



A tiny little proof - via contradiction



A tiny little proof - via contradiction



- Suppose there is a sequence of a fixed number of messages that does the job.

A tiny little proof - via contradiction



- ▶ Suppose there is a sequence of a fixed number of messages that does the job.
- ▶ By assumption this sequence makes generals attack at same time.



A tiny little proof - via contradiction

- ▶ Suppose there is a sequence of a fixed number of messages that does the job.
- ▶ By assumption this sequence makes generals attack at same time.
- ▶ Consider the last such message that was successfully delivered (if not ...).

A tiny little proof - via contradiction

- ▶ Suppose there is a sequence of a fixed number of messages that does the job.
- ▶ By assumption this sequence makes generals attack at same time.
- ▶ Consider the last such message that was successfully delivered (if not ...).
- ▶ Capture this messenger.

A tiny little proof - via contradiction

- ▶ Suppose there is a sequence of a fixed number of messages that does the job.
- ▶ By assumption this sequence makes generals attack at same time.
- ▶ Consider the last such message that was successfully delivered (if not ...).
- ▶ Capture this messenger.
- ▶ The receiver general would decide not to attack (if not ...).

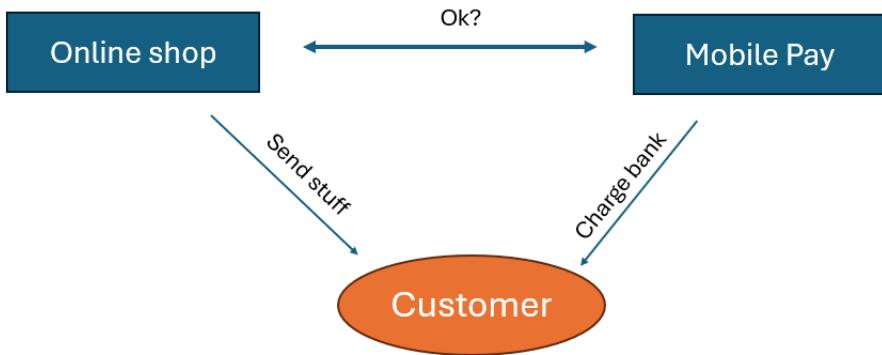
A tiny little proof - via contradiction

- ▶ Suppose there is a sequence of a fixed number of messages that does the job.
- ▶ By assumption this sequence makes generals attack at same time.
- ▶ Consider the last such message that was successfully delivered (if not ...).
- ▶ Capture this messenger.
- ▶ The receiver general would decide not to attack (if not ...).
- ▶ Sender general cannot distinguish the two scenarios, and since his behavior is deterministic he must decide the same before, i.e., attack.

A tiny little proof - via contradiction

- ▶ Suppose there is a sequence of a fixed number of messages that does the job.
- ▶ By assumption this sequence makes generals attack at same time.
- ▶ Consider the last such message that was successfully delivered (if not ...).
- ▶ Capture this messenger.
- ▶ The receiver general would decide not to attack (if not ...).
- ▶ Sender general cannot distinguish the two scenarios, and since his behavior is deterministic he must decide the same before, i.e., attack.
- ▶ We've now created a situation where the suggested protocol leads one general to attack and the other not to attack - a contradiction.

A small application



Two generals vs some muddy children





Two generals vs some muddy children

Or instead (more simple): vs some thirsty mathematicians.

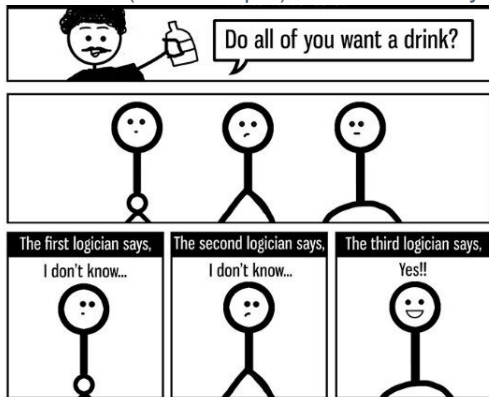


Two generals vs some muddy children

Or instead (more simple): vs some thirsty mathematicians. (Or: synchronus communication)

Two generals vs some muddy children

Or instead (more simple): vs some thirsty mathematicians. (Or: synchronus communication)



Must add timing in the messages



The previous scenario can never work with this type of link.



Must add timing in the messages

The previous scenario can never work with this type of link.

Main issue: At any point in time we never know if the message will eventually come.

Must add timing in the messages

The previous scenario can never work with this type of link.

Main issue: At any point in time we never know if the message will eventually come.

To fix: Add time bounds.

Q1: Which crash/link/timing assumption implements distributed system?

Failure detectors



Failure detectors



A failure detector is just a new module! We can plug it in in any process!

Failure detectors

A failure detector is just a new module! We can plug it in in any process!

Module:

Name: PerfectFailureDetector, **instance** $\mathcal{P}$.

Events:

Indication: $\langle \mathcal{P}, \text{Crash} \mid p \rangle$: Detects that process p has crashed.

Properties:

PFD1: *Strong completeness:* Eventually, every process that crashes is permanently detected by every correct process.

PFD2: *Strong accuracy:* If a process p is detected by any process, then p has crashed.

Example

Implements:

PerfectFailureDetector, **instance** $\mathcal{P}$.

Uses:

PerfectPointToPointLinks, **instance** pl .

upon event $\langle \mathcal{P}, \text{Init} \rangle$ **do**

$alive := \Pi$;

$detected := \emptyset$;

$starttimer(\Delta)$;

upon event $\langle \text{Timeout} \rangle$ **do**

forall $p \in \Pi$ **do**

if $(p \notin alive) \wedge (p \notin detected)$ **then**

$detected := detected \cup \{p\}$;

trigger $\langle \mathcal{P}, \text{Crash} \mid p \rangle$;

trigger $\langle pl, \text{Send} \mid p, [\text{HEARTBEATREQUEST}] \rangle$;

$alive := \emptyset$;

$starttimer(\Delta)$;

upon event $\langle pl, \text{Deliver} \mid q, [\text{HEARTBEATREQUEST}] \rangle$ **do**

trigger $\langle pl, \text{Send} \mid q, [\text{HEARTBEATREPLY}] \rangle$;

upon event $\langle pl, \text{Deliver} \mid p, [\text{HEARTBEATREPLY}] \rangle$ **do**

$alive := alive \cup \{p\}$;

Network latency and bandwidth





Network latency and bandwidth

Physical links: Hard drives in a van, messenger, smoke signals, radio, lasers.

Network links: DSL, cellular data, various wifis, cables, satellites.

► **Latency:** Time until message arrives

1. Hard drives in a van: 1 day
2. intra-continent network cable: 100 ms.

► **Bandwidth:** Data units per time units.

1. Hard drives in a van: 50 TB/1 day = quite a lot.
2. 3G cellular network: 1Mbit/sec

Performance measures



Performance measures



- ▶ SLI = Service Level Indicator.
What are we measuring?



Performance measures

- ▶ SLI = Service Level Indicator.
What are we measuring?
- ▶ SLO = Service Level Objective
How good should it be?



Performance measures

- ▶ SLI = Service Level Indicator.
What are we measuring?
- ▶ SLO = Service Level Objective
How good should it be?
- ▶ SLA = Service Level Agreement



Performance measures

- ▶ SLI = Service Level Indicator.
What are we measuring?
- ▶ SLO = Service Level Objective
How good should it be?
- ▶ SLA = Service Level Agreement
It is an SLO + consequences.

Performance measures

- ▶ SLI = Service Level Indicator.
What are we measuring?
- ▶ SLO = Service Level Objective
How good should it be?
- ▶ SLA = Service Level Agreement
It is an SLO + consequences.

Q: Why should we study these?



- ▶ Measuring means we can improve
- ▶ Spend time improving when it is needed.
- ▶ Reliability is kind of the point with distributed systems.

- ▶ Measuring means we can improve
- ▶ Spend time improving when it is needed.
- ▶ Reliability is kind of the point with distributed systems.

Some SLIs we have seen: bandwidth, latency, fault tolerance, number of messages, uptime, failure detection time.

Reading SLAs *"This solution offers 99% uptime"* Things to consider:

- ▶ How often do we check that a system is up? (Sampling frequency)
- ▶ Do we take blame for network issues?
- ▶ 99% of which time interval?